OR4030 OPTIMIZATION, Chapter 5
Nonlinear Least-Squares Data Fitting
5.1 Introduction

A nonlinear least-squares problem is an unconstrained minimization problem of the form

$$\min_x f(x) = \frac{1}{2}\sum_{i=1}^{m} r_i(x)^2, \tag{1}$$

where the objective function $f(x)$ is defined in terms of auxiliary functions $\{r_i(x)\}$. We call the problem *least squares problem* because we are minimizing the sum of squares of these functions $r_i(x)$.

The main application of such problem is data fitting.

Example 1 Suppose we collected data $\{(t_i, y_i)\}_{i=1}^m$ consisting of the size of a population of antelope at various times. Here t_i corresponds to the time at which the population y_i was counted. Suppose we have the data

$$\begin{array}{rccccc} t_i: & 1 & 2 & 4 & 5 & 8 \\ y_i: & 3 & 4 & 6 & 11 & 20 \end{array}$$

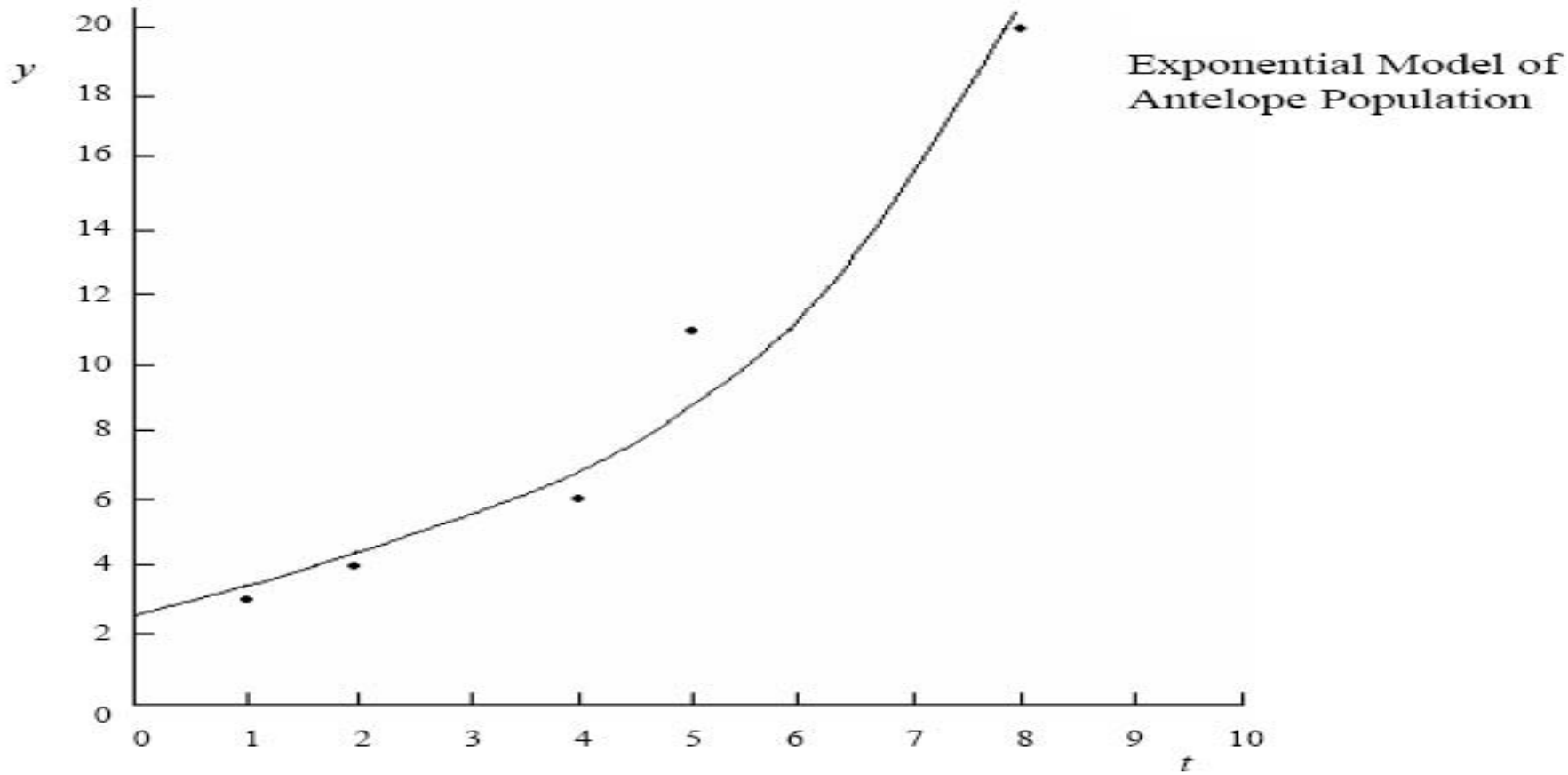
where the times t_i are measured in years, and the populations y_i are measured in hundreds. It is common to model populations using exponential models, so we might hope that

$$y_i \approx x_1 e^{x_2 t_i}$$

for appropriate choices of the parameters x_1 and x_2. The difference between the calculated value of population according to the above formula and the observed population is called the residual which is often expressed as r_i, that is,

$$r_i(x_1, x_2) = x_1 e^{x_2 t_i} - y_i. \tag{2}$$

To say again, r_i is the residual of the i-th data point.



If all $r_i(x_1, x_2) = 0$, $i = 1, 2, 3, 4, 5$, then we would have a perfect fit, that is, the curve of $x_1 e^{x_2 t_i}$ for the selected parameters x_1 and x_2 exactly passes through all data points (t_i, y_i). But often we fail to determine the values of x_1 and x_2 to make all $r_i = 0$. So, to have a best data fitting, we consider the problem

$$\min_{x_1, x_2} f(x_1, x_2) = \frac{1}{2} \sum_{i=1}^{5} r_i(x_1, x_2)^2,$$

which is just a nonlinear least squares problem.

Note that in a least squares problem usually we have more data points than the number of undetermined parameters. In the above example, we have 5 sets of data but only 2 parameters x_1 and x_2 to be determined. So, usually least squares problem is an overdetermined problem.

5.1.1 zero-residual and non-zero residual problems

In a data fitting problem, the value of f is called the (total) residual.

If the minimum value of $f = 0$, i.e., all residuals $r_i(x_1, x_2) = 0$ at the minimum solution (x_1, x_2), $i = 1, 2, \ldots, m$, which means that we can find a set of values for the parameters so that a perfect fit is obtained, the data-fitting problem is called a zero-residual problem.

On the contrary, if the minimum value of $f > 0$, the problem is a non-zero residual problem. In this case, it is impossible to let the curve of the selected model function pass through all data points. In this case, depending on the minimum value of f, we say the data fitting problem is a small residual problem or a big residual problem.

5.1.2 linear and non-linear data fitting problem

As we explained in Chapter 1, if in the expected model, all x_i appear linearly, the data fitting problem is called a linear data fitting problem and the corresponding problem (1) is a linear least squares problem; otherwise we say the problem is a non-linear least squares problem. In the above example, the used model is

$$x_1 e^{x_2 t_i},$$

in which the variables x_1 and x_2 do not appear linearly. So, this is a nonlinear data fitting problem.

If in a data-fitting problem, we suppose the data (t_i, y_i) fit the model

$$y \approx x_1 + x_2 t + x_3 t^2, \tag{3}$$

or the model

$$y \approx x_1 + x_2 \sin t + x_3 \cos t, \tag{4}$$

then these two problems are linear data fitting problems, because the two functions are linear for the variables x_1, x_2 and x_3. Note that in each data point, t takes a particular value t_i. Hence t, t^2, $\sin t$ and $\cos t$ are constants at each t_i. In general, in a linear data fitting problem, the model can be expressed as

$$y \approx a_1 x_1 + a_2 x_2 + \cdots + a_n x_n,$$

where all a_i do not contain any variable x_j $(j = 1, 2, \ldots, n)$. Obviously, linear data fitting model is a special case of nonlinear data fitting problem.

Nonlinear least squares problem (1) is an unconstrained optimization problem. Hence it can be solved by general methods for unconstrained optimization problems. However, due to its special feature, we have more efficient methods to solve this type of optimization problems. Providing such special methods is the main purpose of the chapter.

5.2 Gradient and Hessian Matrix

For the function

$$f(x) = \frac{1}{2}\sum_{i=1}^{m} r_i(x)^2,$$

we now give formulas for the gradient $\nabla f(x)$ and Hessian matrix $\nabla^2 f(x)$. We first let R be the vector-valued residual function:

$$R(x) = (r_1(x), r_2(x), \ldots, r_m(x))^T.$$

For Example 1,

$$R(x) = \begin{bmatrix} x_1 e^{x_2 t_1} - y_1 \\ x_1 e^{x_2 t_2} - y_2 \\ x_1 e^{x_2 t_3} - y_3 \\ x_1 e^{x_2 t_4} - y_4 \\ x_1 e^{x_2 t_4} - y_5 \end{bmatrix} = \begin{bmatrix} x_1 e^{1x_2} - 3 \\ x_1 e^{2x_2} - 4 \\ x_1 e^{4x_2} - 6 \\ x_1 e^{5x_2} - 11 \\ x_1 e^{8x_2} - 20 \end{bmatrix}.$$

For function $R(x)$, let its Jacobian, i.e., its all partial derivatives matrix be $J(x)$:

$$J(x) = \begin{bmatrix} \frac{\partial r_1(x)}{\partial x_1} & \frac{\partial r_1(x)}{\partial x_2} & \cdots & \frac{\partial r_1(x)}{\partial x_n} \\ \frac{\partial r_2(x)}{\partial x_1} & \frac{\partial r_2(x)}{\partial x_2} & \cdots & \frac{\partial r_2(x)}{\partial x_n} \\ \cdots & \cdots & \cdots & \cdots \\ \frac{\partial r_m(x)}{\partial x_1} & \frac{\partial r_m(x)}{\partial x_2} & \cdots & \frac{\partial r_m(x)}{\partial x_n} \end{bmatrix}.$$

A relevant matrix is $\nabla R(x)$ which is defined as:

$$\nabla R(x) = \begin{bmatrix} \nabla r_1(x) & \nabla r_2(x) & \cdots & \nabla r_m(x) \end{bmatrix}.$$

We see that

$$J(x) = \nabla R(x)^T.$$

For Example 1, $r_i(x)$ $(i = 1, 2, 3, 4, 5)$ are given by (2). So, its Jacobian is

$$J(x) = \begin{bmatrix} \frac{\partial r_1(x)}{\partial x_1} & \frac{\partial r_1(x)}{\partial x_2} \\ \frac{\partial r_2(x)}{\partial x_1} & \frac{\partial r_2(x)}{\partial x_2} \\ \cdots & \cdots \\ \frac{\partial r_5(x)}{\partial x_1} & \frac{\partial r_5(x)}{\partial x_2} \end{bmatrix} = \begin{bmatrix} e^{t_1 x_2} & t_1 x_1 e^{t_1 x_2} \\ e^{t_2 x_2} & t_2 x_1 e^{t_2 x_2} \\ e^{t_3 x_2} & t_3 x_1 e^{t_3 x_2} \\ e^{t_4 x_2} & t_4 x_1 e^{t_4 x_2} \\ e^{t_5 x_2} & t_5 x_1 e^{t_5 x_2} \end{bmatrix},$$

where $(t_1, t_2, t_3, t_4, t_5) = (1, 2, 4, 5, 8)$.

We now calculate gradient for the example. As

$$f(x_1, x_2) = \frac{1}{2}\sum_{i=1}^{5}(x_1 e^{x_2 t_i} - y_i)^2,$$

the gradient of f is

$$\begin{aligned}
& \nabla f(x_1, x_2) \\
= & \begin{bmatrix} \sum_{i=1}^{5}(x_1 e^{x_2 t_i} - y_i)e^{x_2 t_i} \\ \sum_{i=1}^{5}(x_1 e^{x_2 t_i} - y_i)x_1 t_i e^{x_2 t_i} \end{bmatrix} \\
= & \begin{bmatrix} e^{x_2 t_1} & e^{x_2 t_2} & e^{x_2 t_3} & e^{x_2 t_4} & e^{x_2 t_5} \\ x_1 t_1 e^{x_2 t_1} & x_1 t_2 e^{x_2 t_2} & x_1 t_3 e^{x_2 t_3} & x_1 t_4 e^{x_2 t_4} & x_1 t_5 e^{x_2 t_5} \end{bmatrix} \begin{bmatrix} x_1 e^{x_2 t_1} - y_1 \\ x_1 e^{x_2 t_2} - y_2 \\ x_1 e^{x_2 t_3} - y_3 \\ x_1 e^{x_2 t_4} - y_4 \\ x_1 e^{x_2 t_5} - y_5 \end{bmatrix} \\
= & J(x)^T R(x).
\end{aligned}$$

In fact it can be verified that the last line gives the general formula for $\nabla f(x)$, because

$$\begin{aligned} \nabla f(x) &= \sum_{i=1}^{m} r_i(x) \nabla r_i(x) \\ &= \nabla R(x) R(x) \\ &= J(x)^T R(x). \end{aligned} \tag{5}$$

For the Hessian matrix,

$$\begin{aligned}
\nabla^2 f(x) &= \sum_{i=1}^{m} [\nabla r_i(x) \nabla r_i(x)^T + r_i(x) \nabla^2 r_i(x)] \\
&= \nabla R(x) \nabla R(x)^T + \sum_{i=1}^{m} r_i(x) \nabla^2 r_i(x) \\
&= J(x)^T J(x) + \sum_{i=1}^{m} r_i(x) \nabla^2 r_i(x), \qquad (6)
\end{aligned}$$

where the first term $J^T J$ contains only the first order derivatives of the residual functions $r_i(x)$, whereas the second term need to use all second order partial derivatives $\frac{\partial^2 r_i}{\partial x_j \partial x_k}$ for $i = 1, \ldots, m$ and $j, k = 1, \ldots, n$.

Return to Example 1, we may verify that

$$
\begin{aligned}
& \nabla^2 f(x_1, x_2) \\
= \quad & J(x)^T J(x) + \sum_{i=1}^{m} r_i(x) \nabla^2 r_i(x) \\
= \quad & \begin{bmatrix} e^{x_2 t_1} & e^{x_2 t_2} & e^{x_2 t_3} & e^{x_2 t_4} & e^{x_2 t_5} \\ x_1 t_1 e^{x_2 t_1} & x_1 t_2 e^{x_2 t_2} & x_1 t_3 e^{x_2 t_3} & x_1 t_4 e^{x_2 t_4} & x_1 t_5 e^{x_2 t_5} \end{bmatrix} \begin{bmatrix} e^{x_2 t_1} & x_1 t_1 e^{x_2 t_1} \\ e^{x_2 t_2} & x_1 t_2 e^{x_2 t_2} \\ e^{x_2 t_3} & x_1 t_3 e^{x_2 t_3} \\ e^{x_2 t_4} & x_1 t_4 e^{x_2 t_4} \\ e^{x_2 t_5} & x_1 t_5 e^{x_2 t_5} \end{bmatrix} \\
& + \sum_{i=1}^{5} (x_1 e^{x_2 t_i} - y_i) \begin{bmatrix} 0 & t_i e^{x_2 t_i} \\ t_i e^{x_2 t_i} & x_1 t_i^2 e^{x_2 t_i} \end{bmatrix}.
\end{aligned}
$$

5.3 Solving Linear Least Squares Problems

We start with the linear problem. For a linear data fitting problem, suppose in the expected formula, there are n parameters $x_1, x_2, \ldots, x_n$ and the formula is

$$y \approx x_1 h_1(t) + x_2 h_2(t) + \cdots + x_n h_n(t).$$

Then the residual of the i-th data point (t_i, y_i) is

$$\begin{aligned} r_i(x) &= h_1(t_i)x_1 + h_2(t_i)x_2 + \cdots + h_n(t_i)x_n - y_i, \\ &= a_{i1}x_1 + a_{i2}x_2 + \cdots + a_{in}x_n - y_i, \quad i = 1, \ldots, m \end{aligned}$$

where

$$a_{ij} = h_j(t_i), \quad i = 1, 2, \ldots, m; \quad j = 1, 2, \ldots, n.$$

So, the Jacobian of the residual vector $R(x)$ is

$$J = \begin{bmatrix} a_{11} & \cdots\cdots & a_{1n} \\ \cdots & \cdots\cdots & \cdots \\ a_{m1} & \cdots\cdots & a_{mn} \end{bmatrix}.$$

We can use an example to explain the constant matrix J. For the linear data fitting formula (3), if five sets of data $(t_1, y_1), \ldots, (t_5, y_5)$ are observed with $t_i = i$, then the Jacobian is

$$J = \begin{bmatrix} 1 & 1 & 1^2 \\ 1 & 2 & 2^2 \\ 1 & 3 & 3^2 \\ 1 & 4 & 4^2 \\ 1 & 5 & 5^2 \end{bmatrix}.$$

From

$$r_i(x) = x_1 + x_2 t_i + x_3 t_i^2 - y_i$$

we see that $R(x) = J(x) - b$, where $b = (y_1, y_2, \ldots, y_m)^T$ is a constant vector. Hence

$$f(x) = \frac{1}{2} \sum_{i=1}^{m} r_i(x)^2 = \frac{1}{2} \|R(x)\|^2 = \frac{1}{2} \|Jx - b\|^2.$$

Using the formulas that we derived for $\nabla f(x)$ and $\nabla^2 f(x)$ and noticing that all $\nabla^2 r_i(x) = 0$, we know that

$$\nabla f(x) = J^T R(x) = J^T (Jx - b);$$

$$\nabla^2 f(x) = J^T J.$$

The stationary point of the linear least squares problem should satisfy the condition:

$$\nabla f(x) = J^T (Jx - b) = 0,$$

i.e.,

$$J^T Jx = J^T b. \tag{7}$$

This equation is called the **normal equation** for linear least squares problem.

$J^T J$ is an $n \times n$ square matrix. As $m > n$, J has more rows than columns. If J has full column rank, that is, if the n columns of J are linearly independent, then $J^T J$ is a positive definite symmetric matrix. Hence the normal equation (7) has a unique solution x^*:

$$x^* = (J^T J)^{-1} J^T b.$$

Since $\nabla^2 f(x^*) = J^T J$ is positive definite, the stationary point x^* is the unique minimum solution to the linear least squares problem. That is, $f(x)$ reaches the minimum value when $x = x^*$. So, when we choose x_i^* as values for the parameters x_i $(i = 1, 2, \ldots, n)$, we obtain the best fitting under the given model.

5.4 Newton's Method and Gauss-Newton Method for Nonlinear Least Squares Problems

We now turn to the nonlinear case. A straightforward method for nonlinear least squares problem is the Newton's method in which we obtain p^k from the equation

$$\nabla^2 f(x^k)\ p = -\nabla f(x^k),$$

where $\nabla f(x^k)$ and $\nabla^2 f(x^k)$ are given by formulas (5) and (6), respectively. Then we take

$$\begin{aligned} x^{k+1} &= x^k + p^k \\ &= x^k - (\nabla^2 f(x^k))^{-1} \nabla f(x^k). \end{aligned}$$

Or we can introduce a line search to ensure global convergence and let

$$x^{k+1} = x^k + \alpha_k p^k,$$

where the step length α_k is determined by a line search rule.

However, due to formula (6), $\nabla^2 f(x^k)$ involves all mn^2 second order partial derivatives $\frac{\partial^2 r_i}{\partial x_j \partial x_k}$ for $i = 1, \ldots, m$ and $j, k = 1, \ldots, n$. Hence it is very time-consuming to obtain $\nabla^2 f(x^k)$ and unless m and n are very small numbers, Newton's method is not practical.

Recall that the right hand side of the formula (6) consists of two parts: the first part $J^T(x)J(x)$ contains only first order partial derivatives and hence easy to compute, but the second part $\sum r_i(x)\nabla^2 r_i(x)$ includes second order partial derivatives of all $r_i(x)$.

We consider an approximation to $\nabla^2 f$:

$$\nabla^2 f(x) = J^T(x)J(x) + \sum_{i=1}^{m} r_i(x)\nabla^2 r_i(x) \approx J^T(x)J(x). \qquad (8)$$

Under two cases the approximation is reasonable.

1. First, all $r_i(x) \approx 0$.

When the model is zero-residual or small-residual, it is the case because $r_i(x^*) \approx 0$ means that for x near x^*, $r_i(x) \approx 0$.

2. Second, all $\nabla^2 r_i(x) \approx 0$.

When all $r_i(x)$ are linear or near linear functions, it is this case.

Gauss-Newton method uses this approximation and calculate the search direction p^k from the equation

$$J^T(x^k)J(x^k)\ p = -J^T(x^k)R(x^k), \tag{9}$$

or to simplify the notation, we write the above equation as

$$J_k^T J_k\ p = -J_k^T R_k,$$

then let

$$\begin{aligned} x^{k+1} &= x^k + \alpha_k p^k \\ &= x^k - \alpha_k (J_k^T J_k)^{-1} J_k^T R_k, \end{aligned}$$

where α_k is determined by a line search.

Example 2 (Newton and Gauss-Newton Methods)

We apply the Gauss-Newton method to the previous exponential model (see Example 1) of the form

$$y_i \approx x_1 e^{x_2 t_i}$$

with data

$$t = (1, 2, 4, 5, 8)^T,$$

$$y = (3.2939, 4.2699, 7.1749, 9.3008, 20.2590)^T.$$

In this example, values of y well fit the model, and hence the problem is a small-residual one and we would expect the Gauss-Newton method to perform much like Newton's method.

Choose an initial point $x^0 = (2.50, 0.25)^T$. At this point, it can be calculated that

$$R(x^0) = \begin{bmatrix} -0.0838 \\ -0.1481 \\ -0.3792 \\ -0.5749 \\ -1.7864 \end{bmatrix} \text{ and } J(x^0) = \begin{bmatrix} 1.2840 & 3.2101 \\ 1.6487 & 8.2436 \\ 2.7183 & 27.18828 \\ 3.4903 & 43.6293 \\ 7.3891 & 147.7811 \end{bmatrix}.$$

Hence

$$\nabla f(x^0) = J^T(x^0)R(x^0) = \begin{bmatrix} -16.5888 \\ -300.8722 \end{bmatrix},$$

$$J^T(x^0)J(x^0) = \begin{bmatrix} 78.5367 & 1335.8479 \\ 1335.8479 & 24559.9419 \end{bmatrix}.$$

The Gauss-Newton search direction p^0 is obtained by solving the linear system

$$J^T(x^0)J(x^0)p = -J^T(x^0)R(x^0).$$

So

$$p^0 = \begin{bmatrix} 0.0381 \\ 0.0102 \end{bmatrix},$$

and if we do not use line search, (in practical use, a line search should be conducted), then the new estimate of the solution is

$$x^1 = x^0 + p^0 = \begin{bmatrix} 2.5381 \\ 0.2602 \end{bmatrix}.$$

We repeat such iterations until $\|\nabla f(x^k)\| \leq 10^{-12}$. The result of complete iterations is given in Table 7.1, and the solution is $x^* = (2.5411, 0.2595)^T$.

For comparison, we now apply Newton's method to the same problem using the same initial point x^0.

$$\begin{aligned}
\nabla^2 f(x^0) &= J^T(x^0)J(x^0) + \sum_{i=1}^{5} r_i(x^0)\nabla^2 r_i(x^0) \\
&= \begin{bmatrix} 78.5367 & 1335.8479 \\ 1335.8479 & 24559.9419 \end{bmatrix} + \sum_{i=1}^{5} r_i(x^0)\nabla^2 r_i(x^0) \\
&= \begin{bmatrix} 78.5367 & 1215.4991 \\ 1215.4991 & 22278.6570 \end{bmatrix}.
\end{aligned}$$

(the matrix is quite close to $J^T J$ which is used in Gauss-Newton method.) The search direction is the solution to the equation

$$\nabla^2 f(x^0)p = -\nabla f(x^0)$$

so that

$$p^0 = \begin{bmatrix} 0.0142 \\ 0.0127 \end{bmatrix} \text{ and } x^1 = x^0 + p^0 = \begin{bmatrix} 2.5142 \\ 0.2627 \end{bmatrix}.$$

The result of complete iterations is given in Table 7.2.

Table 7.1 Gauss-Newton Iterations (Ideal Data)

k	$f(x^k)$	$\|\nabla f(x^k)\|$
0	2×10^{0}	3×10^{2}
1	4×10^{-3}	2×10^{1}
2	2×10^{-8}	3×10^{-2}
3	3×10^{-9}	4×10^{-8}
4	3×10^{-9}	3×10^{-13}

Table 7.2 Newton Iterations (Ideal Data)

k	$f(x^k)$	$\|\nabla f(x^k)\|$
0	2×10^{0}	3×10^{2}
1	1×10^{-1}	5×10^{1}
2	2×10^{-4}	9×10^{-1}
3	5×10^{-9}	6×10^{-3}
4	6×10^{-9}	8×10^{-8}
5	3×10^{-9}	1×10^{-12}

Note that in the example, the performance of Gauss-Newton method is very similar to Newton's method. In fact $f(x^*) \approx 0$ which means that the model is a zero-residual one. In this case, the second part $\sum_{i=1}^{5} r_i(x^k)\nabla^2 r_i(x^k)$ in $\nabla^2 f(x^k)$ is negligible when x^k is close to x^*, and the first part $J^T(x^k)J(x^k)$ is a very good approximation to $\nabla^2 f(x^k)$. That is why Gauss-Newton method is successful in this model.

We now consider the same model

$$y_i \approx x_1 e^{x_2 t_i}$$

but with the data

$$t = (1, 2, 4, 5, 8, 4.1)^T,$$

$$y = (3, 4, 6, 11, 20, 46)^T.$$

Here an extraneous data point (4.1, 46) is added which is an outlier since it is inconsistent with the other data points for this model. So we would expect that the model has a large residual $f(x^*)$ and the performance of the Gauss-Newton method will deteriorate.

The initial point for both Gauss-Newton method and Newton's method is $x^0 = (10, 0.1)^T$. Both methods find the solution $x^* = (9.0189, 0.1206)^T$. But Gauss-Newton method converges slowly. Some iterations are given in Table 7.3 (Gauss-Newton method) and Table 7.4 (Newton's method).

Table 7.4 Newton Iterations (With Outlier)

k	$f(x^k)$	$\|\nabla f(x^k)\|$
0	601.90	2×10^2
1	599.64	1×10^1
2	599.64	2×10^{-2}
3	599.64	6×10^{-8}
4	599.64	5×10^{-13}

Table 7.3 Gauss-Newton Iterations (With Outlier)

k	$f(x^k)$	$\|\nabla f(x^k)\|$
0	601.90	2×10^{2}
1	599.90	8×10^{0}
2	599.67	3×10^{1}
3	599.65	6×10^{0}
4	599.64	2×10^{0}
5	599.64	6×10^{-1}
.	.	.
.	.	.
.	.	.
16	599.64	1×10^{-6}
17	599.64	4×10^{-7}
18	599.64	1×10^{-7}
19	599.64	4×10^{-8}
20	599.64	1×10^{-8}
21	599.64	3×10^{-9}

Advantages and disadvantages of the Gauss-Newton method

Advantages

1. It uses first order derivatives only.
2. For zero-residual, it performs quite like Newton's method and has local quadratical convergence rate.
3. It is quite quickly convergent on problems that are not too nonlinear and have reasonably small residuals.
4. It solves linear least-squares problems in only one iteration.

Disadvantages

1. It is slowly convergent or even not locally convergent on problems that are sufficiently nonlinear or have large residuals.
2. Not well defined if $J(x^k)$ does not have full column rank.

5.5 Levenberg-Marquardt Method

A main disadvantage of Gauss-Newton method is that when $J(x)$ does not have full column rank, $J^T(x)J(x)$ is singular, hence the Gauss-Newton equation (9) is not well defined. To overcome this difficulty, it is suggested that we approximate $\nabla^2 f(x)$ by

$$\nabla^2 f(x) \approx J^T(x)J(x) + \lambda I,$$

where $\lambda > 0$, which is a parameter in the method, and I is the $n \times n$ identity matrix. Note that now the right hand side must be positive definite. So, in the k-th iteration, the equation

$$(J^T(x^k)J(x^k) + \lambda_k I)p = -\nabla f(x^k) \tag{10}$$

is well defined and must have a unique solution p^k. Then we might consider to take $x^{k+1} = x^k + p^k$.

To form a complete algorithm, we need to ask two questions:

(1) Should we always accept $x^k + p^k$ as x^{k+1}?

(2) How to choose the value of λ_k? Should it be a constant in all iterations, or be updated after each iteration?

To answer question (1), we should consider that our purpose in each iteration is to obtain an improved new point, that is, $f(x^{k+1})$ should be smaller than $f(x^k)$. There are several criteria to meet this request, and one of them is to ask

$$f(x^k + p^k) \leq f(x^k) + \beta\langle \nabla f(x^k), p^k \rangle, \tag{11}$$

where β is a given number satisfying $0 < \beta < 1$. Note that as long as $\nabla f(x^k) \neq 0$ (if it is a zero vector, x^k is a stationary point, and we stop computation), from equation (10), we know that $p^k \neq 0$, hence

$$\langle \nabla f(x^k), p^k \rangle = -(p^k)^T (J_k^T J_k + \lambda_k I) p^k < 0$$

(for simplicity, we use J_k to represent $J(x^k)$). Therefore, when (11) holds, $f(x^k + p^k) < f(x^k)$.

We now would ask: does the condition (11) hold? In general, (11) may not be true. However, it can be proved that when λ_k is large enough (11) must hold. The reason can be roughly explained as follows. By Taylor expansion,

$$f(x^k + p^k) \approx f(x^k) + \langle \nabla f(x^k), p^k \rangle + \frac{1}{2}(p^k)^T \nabla^2 f(x^k) p^k. \quad (12)$$

When λ_k is very large, between the two terms $J_k^T J_k$ and $\lambda_k I$, the second one is dominate. Hence the solution of equation (10) is similar to the solution of equation

$$(\lambda_k I)p = -\nabla f(x^k)$$

which is equal to $p = -\nabla f(x^k)/\lambda_k$, that is, $p^k \approx -\nabla f(x^k)/\lambda_k$. Hence when $\lambda_k \to \infty$, $\|p^k\| \to 0$.

In this case the absolute value of the second term in (12) is much larger than what of the third term. Hence when λ_k is large,

$$\frac{1}{2}(p^k)^T \nabla^2 f(x^k) p^k \leq (\beta - 1)\langle \nabla f(x^k), p^k \rangle.$$

Substitute this result into the right hand side of (12), we see that criterion (11) is satisfied.

However, we cannot always let λ_k be a huge number, because

first, when λ_k is large, $\|p^k\|$ is small which means that each step length is very small and hence the method converges very slowly;

second, when λ_k is large, the matrix $J_k^T J_k + \lambda_k I$ will be very much different from $\nabla^2 f(x^k)$ and hence not a good approximation of $\nabla^2 f(x^k)$.

Therefore, to combine these two considerations, a reasonable strategy is that:

if (11) does not hold, we need to increase λ_k and then try again; whereas

if (11) is met, we should consider the possibility to reduce λ_k.

A Levenberg-Marquardt Algorithm

Step 1. Choose a value $\gamma > 1$ and a constant $\beta \in (0, 1)$. Choose an initial iterative point x^1 and an initial parameter value λ_1. Let $k = 1$.

Step 2. If $\nabla f(x^k) \approx 0$, then stop.

Step 3. Solve equation

$$(J_k^T J_k + \lambda_k I)p = -\nabla f(x^k),$$

let the solution be p^k.

Step 4. Check the condition (11):

$$f(x^k + p^k) \leq f(x^k) + \beta\langle \nabla f(x^k), p^k \rangle.$$

If it holds, then let $x^{k+1} = x^k + p^k$, $\lambda_{k+1} = \lambda_k/\gamma$, then go to Step 5; otherwise let $\lambda_k := \gamma\lambda_k$, go to Step 3.

Step 5. Set $k := k + 1$, go to Step 2.

It can be proved that this method globally converges to a point x^* which is at least a stationary point of f.

Levenberg-Marquardt method has many variations, and here we only introduce one of them.

5.6 Structured Secant Method

We may also use secant method to replace Newton's method and to avoid calculating $\nabla^2 f(x^k)$. But a straightforward application of the secant methods of Chapter 4 with the formula $B_{k+1}s_k = y_k$ ($s_k = x^{k+1} - x^k$, and $y_k = \nabla f(x^{k+1}) - \nabla f(x^k)$) to nonlinear least squares is not desirable because these methods approximate all of $\nabla^2 f(x^k)$ by B_k.

As we learned that $\nabla^2 f(x^k)$ consists of two parts, and as the Jacobian $J(x^k)$ should always be calculated in any nonlinear least squares method, the first part $J^T(x^k)J(x^k)$ is readily available. Hence we only need to approximate the second part of $\nabla^2 f(x^k)$, which for convenience is denoted by $T(x^k)$:

$$T(x^k) = \sum_{i=1}^{m} r_i(x^k)\nabla^2 r_i(x^k). \tag{13}$$

We try to use a matrix A_k to approximate $T(x^k)$, or equivalently use a matrix A_{k+1} to approximate $T(x^{k+1})$. Let matrix H_i^{k+1} be an approximation to $\nabla^2 r_i(x^{k+1})$ which satisfies the secant equation for function $r_i(x)$, that is,

$$\begin{aligned} & H_i^{k+1}(x^{k+1} - x^k) \\ = \quad & \nabla r_i(x^{k+1}) - \nabla r_i(x^k) \\ = \quad & [\text{row } i \text{ of } J(x^{k+1})]^T - [\text{row } i \text{ of } J(x^k)]^T. \end{aligned} \tag{14}$$

From (13) and (14),

$$
\begin{aligned}
& A_{k+1}(x^{k+1} - x^k) \\
\approx \quad & \sum_{i=1}^{m} r_i(x^{k+1}) \nabla^2 r_i(x^{k+1})(x^{k+1} - x^k) \\
\approx \quad & \sum_{i=1}^{m} r_i(x^{k+1}) H_i^{k+1}(x^{k+1} - x^k) \\
= \quad & \sum_{i=1}^{m} r_i(x^{k+1}) \{[\text{row } i \text{ of } J(x^{k+1})]^T - [\text{row } i \text{ of } J(x^k)]^T\} \\
= \quad & J(x^{k+1})^T R(x^{k+1}) - J(x^k)^T R(x^{k+1}) \\
\stackrel{def}{=} \quad & y_k^{\#}. \qquad (15)
\end{aligned}
$$

Let $s_k = x^{k+1} - x^k$, then the above structured secant equation can be expressed as

$$A_{k+1} s_k = y_k^{\#}. \tag{16}$$

To summarize, once we have x^k and x^{k+1}, we can obtain $R(x^k)$, $R(x^{k+1})$, $J(x^k)$, and $J(x^{k+1})$. We then calculate s_k and $y_k^{\#}$, and find a matrix A_{k+1} that satisfies the structured secant equation (16). Now

$$\nabla^2 f(x^{k+1}) \approx J(x^{k+1})^T J(x^{k+1}) + A_{k+1},$$

and we obtain p^{k+1} from the approximate Newton equation

$$[J(x^{k+1})^T J(x^{k+1}) + A_{k+1}]p = -\nabla f(x^{k+1}),$$

and the next iterative point x^{k+2} by

$$x^{k+2} = x^{k+1} + p^{k+1}$$

if no line search is made, or

$$x^{k+2} = x^{k+1} + \alpha_{k+1} p^{k+1}$$

if a line search is made to decide the step length α_{k+1}.

Note that on the right hand side of (16), $y_k^{\#}$ is not equal to $y_k = \nabla f(x^{k+1}) - \nabla f(x^k) = J(x^{k+1})^T R(x^{k+1}) - J(x^k)^T R(x^k)$. This difference is reasonable because now A_{k+1} is not an approximation of the whole $\nabla^2 f(x^{k+1})$, but only an approximation to the second part of $\nabla^2 f(x^{k+1})$. Hence we should not expect the right hand side of the above secant equation to be y_k.

As the above method approximates only a part of the Hessian matrix by taking advantage of the special structure of the problem, we call such secant method a structured secant method, or a structured quasi-Newton method.

Let us return to the structured secant equation (16). If $n = 1$, there is only one parameter x_1 to be determined. In this case the two sides of (16) are scalars and hence A_{k+1} is a number which can be fully determined by the formula

$$A_{k+1} = \frac{[J(x^{k+1}) - J(x^k)]^T R(x^{k+1})}{x^{k+1} - x^k},$$

with an initial guess of the parameter value x^1 and an initial value A_1 to approximate $\sum_{i=1}^{m} r_i(x^1) r_i''(x^1)$. We give such an example:

Example 3 Suppose the expected model is

$$y \approx e^{tx_1},$$

and three sets of data are observed:

$$(t_1, y_1) = (1, 2), \qquad (t_2, y_2) = (2, 4), \qquad (t_3, y_3) = (3, y_3),$$

where we discuss several different values for y_3. In this example, $n = 1$ and $m = 3$. We use three different methods: Gauss-Newton method, Newton's method, and the structured secant method, to solve the nonlinear least squares problem with the same initial value x^1. We stop computation until $\|\nabla f(x^k)\| \leq 10^{-10}$.

In the following table,

the first column gives five different values for y_3 which makes the problem a zero-residual, small-residual, or large-residual problem;

the second column gives two different initial guesses of the value x^1;

the columns 3-5 show respectively the number of iterations for each of the three methods to meet the termination requirement, where * means that the method fails to be convergent after a large number of iterations;

the sixth column shows the optimal solution x^*, and

the last column gives the value of residual $f(x^*)$.

In all computation, we choose $A_1 = 1$ for the structured secant method.

Table 7.5 Comparison of Three Methods on a Small Problem

		number of iterations required				
y_3	x^1	Gauss-Newton	Newton	Secant	x^*	$f(x^*)$
8	1	5	7	6	0.69315	0
	0.6	4	6	5		
3	1	12	9	8	0.44005	1.6390
	0.5	9	5	4		
-1	1	34	10	11	0.044744	6.9765
	0	32	4	5		
-4	1	*	12	13	-0.37193	16.435
	-0.3	*	4	6		
-8	1	*	12	15	-0.79148	41.145
	-0.7	*	4	8		

* — fail to converge

From the table we see that for zero-residual problems, Gauss-Newton method performs comparably, but when the residual becomes large (with $f(x^*) = 6.9765$ or even bigger), its performance deteriorates, and finally it does not work.

On the other hand, we see that overall, the secant method need more or less the same, or only a bit more, iterations than that Newton's method required.

Note that in this example we compare only the number of iterations, and if the time of computation in each iteration is concerned, the three methods are very different because Newton's method need to calculate second order derivatives whereas Gauss-Newton method and the secant method requires only the first order partial derivatives.

When $n > 1$, A_{k+1} is not a scalar, but an $n \times n$ matrix. In this case equation (16) can not completely specify A_{k+1}, and many different A_{k+1} would satisfy the structured secant equation. We can determine A_{k+1} using the techniques given in Chapter 4.

Since $T(x^{k+1})$ is symmetric, A_{k+1} should be too; however in general $T(x)$ may not be positive definite even at the solution x^* (because $T(x)$ is only a part of $\nabla^2 f(x)$), so we will not require $A(x)$ to be positive definite.

Several update formulas are proposed, and the following one, suggested by Dennis, Gay and Welsch (1981), has been recognized with good performance:

$$A_{k+1} = A_k + \frac{(y_k^{\#} - A_k s_k) y_k^T + y_k (y_k^{\#} - A_k s_k)^T}{y_k^T s_k} - \frac{(y_k^{\#} - A_k s_k)^T s_k y_k y_k^T}{(y_k^T s_k)^2}. \tag{17}$$